



ECA304

PROGRAMAÇÃO

ORIENTADA A OBJETOS

Alexandre H. MOREIRA

CLASSES E OBJETOS - ENCAPSULAMENTO



Introdução

Encapsulamento é o princípio de esconder os detalhes internos de um objeto e controlar como seus dados são acessados ou modificados.

"Encapsular é proteger. Só quem tem autorização pode acessar ou alterar os dados."



□ Por que usar Encapsulamento?

- Proteger o estado interno do objeto.
- Evitar uso indevido ou mudanças indesejadas.
- Facilitar a manutenção e evolução do código.



Exemplo sem Encapsulamento (problema)

```
class Conta:  
    def __init__(self, saldo):  
        self.saldo = saldo  
  
conta = Conta(1000)  
conta.saldo = -500  
print(conta.saldo)
```



Introdução



Exemplo com Encapsulamento (solução)

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo # atributo privado

    def ver_saldo(self):
        return self.__saldo

    def depositar(self, valor):
        if valor > 0:
            self.__saldo += valor

    def sacar(self, valor):
        if 0 < valor <= self.__saldo:
            self.__saldo -= valor
```

```
conta = Conta(1000)
conta.depositar(200)
conta.sacar(300)
print(conta.ver_saldo())
```

900



PÚBLICO, PROTEGIDO E PRIVADO

□ Considere o código a seguir:

```
class minhaClasse():  
    def __init__(self):  
        self.publico = "Eu sou publico."  
        self._protegido = "Eu sou protegido."  
        self.__privado = "Eu sou privado."  
  
teste = minhaClasse()  
print(teste.publico)  
print(teste._protegido)  
print(teste.__privado)
```

□ E sua saída:

Eu sou publico.

Eu sou protegido.

Traceback (most recent call last):

File "aula_09.py", line 17, in <module>

print(teste.__privado)

AttributeError: 'minhaClasse' object has no attribute '__privado'



PÚBLICO, PROTEGIDO E PRIVADO – RESUMINDO

Classes e Objetos - Encapsulamento

Nome	Tipo	Significado
nome	Público	Pode ser utilizado livremente dentro ou fora de uma definição de classe.
_nome	Protegido	Não deve ser utilizado fora da definição de classe, a menos que dentro de uma definição de subclasse.
__nome	Privado	Inacessível e invisível. Não é possível ler nem escrever nesses atributos, exceto dentro da própria definição de classe.



MÉTODOS *SETTER* E *GETTER*

- Podemos definir os métodos getter e setter como:
 - Um método que permite acessar um atributo;
 - Um método que permite alterar o valor de um atributo.

MÉTODOS SETTER E GETTER



- Considere o código a seguir:

Getter

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo # atributo privado

    def ver_saldo(self):
        return self.__saldo

    def depositar(self, valor):
        if valor > 0:
            self.__saldo += valor

    def sacar(self, valor):
        if 0 < valor <= self.__saldo:
            self.__saldo -= valor
```

Setter

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo # atributo privado

    def ver_saldo(self):
        return self.__saldo

    def depositar(self, valor):
        if valor > 0:
            self.__saldo += valor

    def sacar(self, valor):
        if 0 < valor <= self.__saldo:
            self.__saldo -= valor
```




PRIVADO PERO NO MUCHO!

- Acabamos de verificar que quando adicionamos “__” atributo o tornamos privado, porém, é possível driblar isso:

```
Class minhaClasse(object):  
    def __init__(self):  
        self.publico = "Eu sou publico."  
        self._protegido = "Eu sou protegido."  
        self.__privado = "Eu sou privado."  
  
teste = minhaClasse()  
print(teste._minhaClasse__privado)
```

Eu sou privado.

- Não existem métodos ou atributos privados em Python!



Boas práticas

Note: Python doesn't have the notion of **access modifiers**, such as `private`, `protected`, and `public`, to restrict access to attributes and methods in a class. In Python, the distinction is between **public** and **non-public** class members.

If you want to signal that a given attribute or method is non-public, then you should use the well-established Python **convention** of prefixing the name with an underscore (`_`).

Note that this is just a convention. It doesn't stop you and other programmers from accessing the attributes using **dot notation**, as in `obj._attr`. However, it's bad practice to violate this convention.



EXERCÍCIO PARA ENTREGA

A CLASSE TEMPERATURA



Exercício para entrega

- ❑ Criar uma classe **Temperatura** que:
 - ❑ Armazene a temperatura atual (inicial em 25°C).
 - ❑ Não permita valores abaixo de -273.15°C (zero absoluto).
 - ❑ Use métodos **get_temperatura()** e **set_temperatura(valor)**.

```
# Exemplo de uso esperado:  
t = Temperatura()  
print(t.get_temperatura()) # deve mostrar 25  
t.set_temperatura(-500)    # não deve aceitar  
t.set_temperatura(30)      # deve aceitar  
print(t.get_temperatura()) # deve mostrar 30
```